Documentation for the simple non-hydrostatic model nh_simple

nh_simple is a finite-difference sibling of the pseudo-spectral model ps3D: it integrates the same scaled non-hydrostatic Boussinesq equations on the same kind of domain (horizontally always periodic, vertically periodic or bounded by rigid lids), but discretises them on an Arakawa C grid in physical space and uses the Fourier transform only for the pressure inversion. The LES closures are ported from ps3D. Within pyOM3, nh_simple is one of the ocean components that can be coupled to the spectral surface wave model wim_simple; the hooks of the Lagrangian-mean formulation of that coupling are noted where they appear.

Continuous equations

The scaled non-hydrostatic Boussinesq equations (see the scaling section of the ps3D documentation for the derivation) are

$$\partial_t \boldsymbol{u} + f \underset{\neg}{\boldsymbol{u}} + \boldsymbol{\nabla} p = -Ro\,(\boldsymbol{u}\cdot\boldsymbol{\nabla}\boldsymbol{u} + w\partial_z \boldsymbol{u}) \quad , \quad \partial_t b + N^2 w = -Ro\,(\boldsymbol{u}\cdot\boldsymbol{\nabla} b + w\partial_z b)$$
$$\delta^2 \partial_t w + \partial_z p - b = -Ro\,\delta^2\,(\boldsymbol{u}\cdot\boldsymbol{\nabla} w + w\partial_z w)$$

with the diagnostic relation $\boldsymbol{\nabla}\cdot\boldsymbol{u}+\partial_z w = 0$. Vectors are horizontal, $\underset{\neg}{\boldsymbol{u}}$ denotes anticlockwise rotation by 90°, Ro is the Rossby number and $\delta = H/L$ the aspect ratio; f and the background stability frequency N are constants (f-plane, uniform stratification). The buoyancy b is the deviation from the background stratification $N^2 z/Ro$; the term $N^2 w$ is the advection of this background. Kinetic and potential energy are $\boldsymbol{u}^2/2 + \delta^2 w^2/2$ and $b^2/2N^2$. Setting $Ro = 1$ and $\delta^2 = 1$ recovers the plain dimensional equations, which is the configuration required by the LES closures below.

Grid and spatial discretisation

All fields live on an Arakawa C grid with uniform spacings $\Delta x, \Delta y, \Delta z$: b and p at cell centres, u, v, w on the respective cell faces. The momentum advection is the second-order energy-conserving flux form, e.g. for u

$$(\partial_t u)_{adv} = -Ro\left(\delta_x^- \overline{\overline{u}^{x+}\overline{u}^{x+}} + \delta_y^- \overline{\overline{u}^{y+}\overline{v}^{x+}} + \delta_z^- \overline{\overline{u}^{z+}\overline{w}^{x+}}\right)$$

in the finite-difference and averaging operator notation of the ps3D documentation ($\delta^{\pm}$ forward/backward differences, $\cdot^{\pm}$ two-point averages), and analogously for v and w (the w tendency carries the factor δ^2). The Coriolis force uses the standard four-point average of the C grid.

For the buoyancy advection three flux-form schemes are available, selected by attribute: the second-order centred scheme (default, energy conserving), the second-order scheme with Superbee flux limiter (`buoyancy_superbee_advection`, from MITgcm; monotone,

and its implicit scalar mixing can replace the explicit SGS buoyancy diffusion of the LES closures, see below), and the third-order upwind scheme (`buoyancy_upwind3_advection`, from MITgcm). Momentum advection is always the energy-conserving second-order form.

Pressure solver

The velocity is first advanced with all terms except the pressure gradient to an intermediate value $\boldsymbol{u}^*, w^*$; the pressure then enforces the divergence constraint,

$$\left(\delta_x^- \delta_x^+ + \delta_y^- \delta_y^+ + \delta_z^- \delta_z^+ / \delta^2\right) p = \frac{1}{\Delta t} \left(\delta_x^- u^* + \delta_y^- v^* + \delta_z^- w^*\right)$$

followed by the projection $\boldsymbol{u} = \boldsymbol{u}^* - \Delta t \, \boldsymbol{\nabla}^+ p$ and $w = w^* - (\Delta t / \delta^2) \, \delta_z^+ p$. As shown in the ps3D documentation, the Fourier transform of the discrete one-dimensional operator $\delta^- \delta^+$ is $-\hat{k}^+ \hat{k}^- = -2\,(1 - \cos k\Delta)\,/\Delta^2$, which approaches $-k^2$ for $k\Delta \to 0$; the discrete Poisson equation is therefore diagonal in spectral space with the eigenvalue $\hat{k}^2 = \hat{k}_x^+ \hat{k}_x^- + \hat{k}_y^+ \hat{k}_y^- + \hat{k}_z^+ \hat{k}_z^- / \delta^2$, so the discrete divergence of the projected velocity vanishes to round-off.

Two cases are implemented, selected by `periodic_in_z`. For the triple-periodic domain a 3D FFT (parallel, through the p3dfft library) inverts the full eigenvalue; the mean mode $\hat{k}^2 = 0$ is set to zero. For rigid lids at top and bottom, a 2D FFT in x, y is combined with a tridiagonal solve in z (Thomas algorithm) with Neumann boundary conditions at the lids; the horizontal-mean column ($\hat{k}_x = \hat{k}_y = 0$), for which the tridiagonal system is singular, is integrated directly by the second-order recurrence $p_{k+1} = 2p_k - p_{k-1} + \delta^2 \Delta z^2 f_k$ with $p_0 = 0$.

Boundary conditions

The domain is always periodic in x and y (halo wrap or MPI exchange). In z it is either periodic (`periodic_in_z` true) or bounded by rigid free-slip lids: all vertical fluxes — advective, frictional, diffusive and the LES fluxes — are set to zero on the top and bottom faces, and w vanishes there.

Friction and mixing

Harmonic friction with constant coefficients acts on the momentum when the attributes are present: with both `Ah` and `Av` the full operator $A_h \boldsymbol{\nabla}_h^2 + A_v \partial_{zz}$, with only one of them the corresponding horizontal-only or vertical-only part. Harmonic buoyancy mixing requires both `Kh` and `Kv`. Biharmonic friction and mixing ($-A_{hbi} \boldsymbol{\nabla}_h^4$ etc.) are activated by the pairs `Ahbi`, `Avbi` for momentum and `Khbi`, `Kvbi` for buoyancy. All these operators are applied with a forward Euler step outside the Adams–Bashforth interpolation of the adiabatic tendencies, so their coefficients are subject to the usual explicit diffusive stability limits.

Time stepping

The adiabatic tendencies (advection, Coriolis, buoyancy force) are interpolated with the multi-step Adams–Bashforth scheme,

$$(c_A, c_B, c_C) = (1, 0, 0) \;\rightarrow\; \left(\frac{3}{2}, -\frac{1}{2}, 0\right) \;\rightarrow\; \left(\frac{23}{12}, -\frac{16}{12}, \frac{5}{12}\right)$$

i.e. a forward Euler start, one AB2 step and third-order Adams–Bashforth from the third step on, with the tendencies held in a rotating three-level buffer. Friction, mixing and the LES closures update the fields directly with forward Euler before the Adams–Bashforth increment; the pressure projection concludes the step. A user-defined forcing can be added to the tendencies at the current time level through the optional method `set_forcing(tau)` (it then enters the Adams–Bashforth interpolation), or applied as a direct increment through `set_forcing_direct()` after the Adams–Bashforth update and before the pressure solve. The latter bypasses the AB weights and is the hook for the Stokes tendency $\Delta \boldsymbol{u}_S$ of the Lagrangian-mean ($\boldsymbol{u}_L$) formulation of the wave coupling, which must be a plain forward increment and must precede the projection so that it acts on its divergence.

LES closures

The LES closures of ps3D (`les_closures.f90`) are ported as the mixin class `les` (module `les.py`): the Smagorinsky closure, the filtered-structure-function closure of Ducros et al. (1996), and a Deardorff-type prognostic TKE closure, plus an optional Peclet bound on the horizontal viscosities. The closures are formulated dimensionally and refuse to run unless $Ro = 1$ and $\delta^2 = 1$. One call of `les_closure()` per time step (after the harmonic friction, before the Adams–Bashforth increment, e.g. hooked through `set_forcing`) computes the strain-rate tensor on its natural C-grid positions, evaluates the enabled closure(s), and applies the anisotropic viscous and diffusive flux divergences to u, v, w and b with a forward Euler step.

Smagorinsky (`enable_smagorinsky`): $\nu = (c_s\Delta)^2\sqrt{2S_{ij}S_{ij}}$ with $\Delta = (\Delta x \Delta y \Delta z)^{1/3}$ and $c_s =$ `smag_coeff`; the diffusivity is ν/Pr with the turbulent Prandtl number `prandtl`. The viscosity is evaluated separately at the four staggered positions where its fluxes are needed (cell centre, horizontal corner, above u, above v).

Ducros et al. (1996) (`enable_ducros_96`): the viscosity is built from the second-order structure function of the high-pass filtered velocity (two iterated horizontal Laplace filters, three with `enable_three_level_filter`).

Prognostic TKE (`enable_tke_closure`): a Deardorff-type closure with prognostic subgrid TKE on the w grid, a stratification-limited length scale $\ell = \min(\Delta,\, 0.76\sqrt{e/N^2})$, dissipation $(0.19 + 0.51\ell/\Delta)\, e^{3/2}/\ell$, TKE diffusion with coefficient `c_eke` times the eddy diffusivity, and shear and buoyancy production diagnosed from the actual discrete mixing fluxes of the previous step. Viscosity and diffusivity are $\nu = c_m \ell \sqrt{e}$ and $\kappa = (1 + 2\ell/\Delta)\,\nu$ with $c_m =$ `c_m`; N^2 always includes the background N_0^2.

Stability caps. All closures cap their viscosities and diffusivities against the explicit diffusive limit of the forward Euler update. The caps are direction-aware: each staggered viscosity is capped with the most restrictive grid spacing among the directions its fluxes act in (e.g. the cell-centre viscosity by $\min(\Delta x, \Delta y, \Delta z)^2/(4\Delta t)$). This deviates deliberately from the original ps3D caps, which use geometric means of the spacings and are valid only for near-isotropic grids: for $\Delta x \gg \Delta z$ they admit vertically unstable viscosities.

Boundary treatment. With rigid lids the vertical shear seen by the closures is zeroed on the lids (`les_freeslip_strain`, default on), consistent with the free-slip zero-flux condition of the mixing; otherwise the zero halo above the lid produces a spurious shear that inflates the viscosity in the top and bottom cells.

Scalar mixing. The explicit SGS buoyancy diffusion can be switched off (`les_mix_b`) when a flux-limited buoyancy advection scheme (Superbee) provides the scalar SGS mixing implicitly; under the TKE closure the resolved buoyancy production is then zero.

Peclet bound (`enable_peclet_bound`): where the grid Peclet number of the resolved flow exceeds `Peclet_limit`, the horizontal viscosities are raised proportionally. The bound uses the prognostic (advecting) velocity, i.e. $\boldsymbol{u}_L$ in the Lagrangian-mean formulation.

Coupled wave–ocean setups. When the prognostic velocity is the Lagrangian mean $\boldsymbol{u}_L$, the attribute `les_mix_eulerian` makes the strains and mixing fluxes act on the Eulerian velocity $\boldsymbol{u} = \boldsymbol{u}_L - \boldsymbol{u}_S$ (attributes `u_S`, `v_S` and optionally `w_S` supplied by the coupler), with the tendency added to the prognostic $\boldsymbol{u}_L$ — the down-gradient closure parameterises Eulerian fluctuations, so mixing $\boldsymbol{u}_L$ would spuriously diffuse the Stokes shear. This is the LES counterpart of the `mix_eulerian` option of the TKE closure in pe_simple.

Domain decomposition

The parallelisation follows the pencil decomposition of the p3dfft library used by the pressure solver: the x direction is kept local on every processor, and the constructor arguments `n_pes_x` and `n_pes_y` define the processor grid over the y and z directions (internally the blocks are addressed as y/z blocks). Model arrays are Fortran-ordered (N_x, N_y, N_z) with halo points on all sides; the pressure solver requires this ordering. Halo exchanges wrap periodically in x and y; in z they wrap only for the triple-periodic case.

Snapshot diagnostics

Two snapshot classes live in the module `snapshot.py`; either replaces the model class as the base of the configuration and overrides `setup` and `diagnose`. Both read the attribute `snapint` (snapshot interval in seconds, required), take the setup arguments `snap_file_name` and `show_each_time_step` (default on: print the model time every step; off: only at output steps), and write at $n = 0$ and whenever $n \bmod (\text{snapint}/\Delta t) = 0$. Both record the variables u, v, w, b, p on their staggered coordinate axes (`xt`/`yt`/`zt` for cell centres and `xu`/`yu`/`zu` for the faces, shifted by half a spacing); the vertical axis is negative downwards with the surface at $z = 0$ and the cell centres at $z = -\Delta z/2, \ldots$. Halo points are stripped before writing.

snap_nc4 writes a single netCDF file (default `'snap.cdf'`). At start-up it detects whether the netCDF4/HDF5 build supports parallel I/O (`__has_parallel4_support__` or pnetcdf) and reports the result. In parallel mode the file is created and opened collectively by all PEs, and every PE writes its subdomain into the global arrays in a single collective pass per snapshot. Without parallel support the original serialised path is used: PE0 creates the file and extends the time dimension, then each PE in turn opens the file and writes its block, synchronised with barriers — correct on any build, but increasingly slow with many PEs. The internal Fortran ordering is transposed on the fly, so the file layout (time, z, y, x) is the same in both modes.

snap_zarr_gather writes a single zarr store (default `'snap.zarr'`) with field arrays of shape (time, n_x, n_y, n_z), always x-first regardless of the internal ordering, one time slice per chunk. The design goal is a Lustre-friendly file count: with the default `z_chunks` = 1 each snapshot adds one chunk file per field — of the order of six files per snapshot regardless of the number of PEs — instead of the one-file-per-PE-per-field fragmentation of a per-block chunking, which can overwhelm the metadata server of a parallel file system at large PE counts. Every PE participates in the collective gather of each field (`gather_array_mpi`), but only PE0 touches the store, writing a full horizontal slab of $\lceil n_z/$`z_chunks`$\rceil$ levels per chunk; increase `z_chunks` only if a single field slab is too large to hold and write in one piece. Fields are gathered and written one at a time to bound the PE0 memory to one global field. The time axis and the field arrays are resized (appended) per snapshot, so the store remains valid after an interrupted run; an existing store of the same name is removed at start-up. The on-disk format is pinned to zarr v2 even when a zarr-python v3 library writes it, so the output stays readable by v2 environments; the cosmetic consolidated-metadata warning of v3 is silenced. Compared to the netCDF path this trades the gather (one global field on PE0) for independence from parallel-I/O library support.

Model parameters and options

All parameters and options are attributes of the model instance, set either through the parameter dictionary passed at construction or by assigning the attribute before the time loop; options with a default may be left unset.

Required parameters (no default)

Attribute	Unit	Meaning
`nx`, `ny`, `nz`	—	global grid size (cells, without halo points)
`dx`, `dy`, `dz`	m	grid spacing (uniform)
`dt`	s	time step
`f0`	1/s	Coriolis parameter (f-plane)
`N0`	1/s	background stability frequency

Attribute	Unit	Meaning
`Ro`	—	Rossby number of the scaled equations (1 for the dimensional form; the LES closures require 1)
`dsqr`	—	squared aspect ratio δ^2 (1 for the fully non-hydrostatic dimensional form; the LES closures require 1)
`periodic_in_z`	—	triple-periodic domain if true; rigid free-slip lids at top and bottom if false. x and y are always periodic

The constructor additionally takes `n_pes_x`, `n_pes_y` (processor grid over the y and z pencils, default 1×1; their product must equal the communicator size), `halo_points` (default 1) and `comm_in` (default: MPI world communicator).

Forcing hooks and coupling fields

Attribute / method	Meaning
`set_forcing(tau)`	optional user method; adds forcing to the tendencies at time level `tau` (enters the Adams–Bashforth interpolation). Also the natural hook to call `les_closure()`
`set_forcing_direct()`	optional user method; direct increment after the Adams–Bashforth update and before the pressure solve, bypassing the AB weights. Used for the Stokes tendency $\Delta \boldsymbol{u}_S$ of the Lagrangian-mean ($\boldsymbol{u}_L$) formulation
`u_S`, `v_S`, `w_S`	Stokes velocity from the wave model (w_S optional); required by `les_mix_eulerian`

Options with defaults

Attribute	Default	Meaning
`buoyancy_superbee_advection`	off	buoyancy advection with the Superbee flux limiter instead of the centred second-order scheme
`buoyancy_upwind3_advection`	off	third-order upwind buoyancy advection
`Ah`, `Av`	unset	harmonic viscosities (m^2/s); either one alone activates the horizontal-only / vertical-only operator
`Kh`, `Kv`	unset	harmonic buoyancy diffusivities (m^2/s); both must be present

Attribute	Default	Meaning
`Ahbi`, `Avbi`	unset	biharmonic viscosities (m^4/s); both must be present
`Khbi`, `Kvbi`	unset	biharmonic buoyancy diffusivities (m^4/s); both must be present

LES options (read at `init_les`)

Attribute	Default	Meaning
`enable_smagorinsky`	off	Smagorinsky closure
`enable_ducros_96`	off	Ducros et al. (1996) structure-function closure
`enable_tke_closure`	off	Deardorff-type prognostic TKE closure
`enable_three_level_filter`	off	three instead of two iterated Laplace filters (Ducros closure)
`enable_peclet_bound`	off	raise horizontal viscosities where the grid Peclet number exceeds `Peclet_limit`
`les_mix_eulerian`	off	strains and mixing act on the Eulerian velocity $\boldsymbol{u}_L - \boldsymbol{u}_S$ ($\boldsymbol{u}_L$ formulation)
`les_freeslip_strain`	on	zero the vertical shear seen by the closure at the rigid lids
`les_mix_b`	on	explicit SGS buoyancy diffusion; switch off when a flux-limited advection scheme provides the scalar SGS mixing
`smag_coeff`	0.148	Smagorinsky coefficient
`prandtl`	0.47	turbulent Prandtl number
`min_smag`	0	viscosity floor (m^2/s)
`Peclet_limit`	50	grid Peclet number bound
`c_m`	0.1	TKE-closure viscosity constant
`c_eke`	2	TKE diffusion constant
`tke_min`	10^{-8}	TKE floor (m^2/s^2)
`les_epsln`	10^{-20}	numerical floor of the closure

Snapshot diagnostics

Both snapshot classes (module `snapshot.py`) read the attribute `snapint` (snapshot interval in seconds, required) and take the setup arguments `snap_file_name` (defaults `'snap.cdf'` / `'snap.zarr'`) and `show_each_time_step` (default on); `snap_zarr_gather` additionally takes `z_chunks` (default 1: one chunk file per field and snapshot; larger values split the vertical into $\lceil n_z/$`z_chunks`$\rceil$-level chunks).